

Файли

Функції open() і close(). Інструкція with

Для роботи з файлами в Python призначена вбудована функція open(), яка створює об'єкт файлу на основі переданих їй імені файлу, режиму роботи з ним (читання, запис тощо) та (не обов'язково) методу кодування. По завершенні роботи з файлом його потрібно закрити методом close(). Незакриття файлу може призвести до втрати даних, а метод close() гарантує, що дані будуть записані у файл.

```
f = open('filename', 'w', encoding="utf-8")
... # Дії над файлом
f.close()
```

Замість цього можна (і рекомендується) використовувати інструкцію with, яка автоматично закриває файл по завершенню виконання блоку інструкцій:

```
with open('filename', 'w', encoding="utf-8") as f:
    ... # Дії над файлом
```

Дані, які зчитуються з текстового файлу і записуються у файл, завжди є рядками (тип str). Дані з інших типів файлів зчитуються по байтах.

Запис у файл

Щоб створити об'єкт файлу для подальшого запису в нього, потрібно передати функції open() першим аргументом ім'я цього файлу, а другим — рядок 'w' (від write).

Інтерпретатор шукатиме файл result.txt у тій же папці, де знаходиться програма. Якщо файлу з таким іменем немає, то він буде створений. Якщо файл з таким іменем є, то (увага!) усі дані в ньому будуть стерті без попередження. Тому, щоб відкрити існуючий файл в режимі "додавання в кінець", замість 'w' другим аргументом функції open() потрібно передати 'a' (від append).

Для запису у файл використовується метод write().

```
with open('result.txt', 'w') as f:
    f.write('Квадрати перших десяти натуральних чисел:\n')
    for i in range(1, 11):
        f.write(str(i**2) + '\t')
```

```
# Вміст файлу result.txt після виконання програми:  
Квадрати перших десяти натуральних чисел:  
1 4 9 16 25 36 49 64 81 100
```

Читання з файлу

Для відкриття файлу з метою подальшого зчитування даних з нього потрібно передати функції `open()` другим аргументом `'r'` (можна не передавати - це значення використовується за замовчуванням):

```
f = open('result.txt', 'r')
```

1) Читання усього вмісту файлу в один рядок:

```
all_content = f.read()
```

Якщо намагатися ще раз виконати попередню інструкцію, то отримаємо порожній рядок, тому для повторного зчитування даних з файлу його треба заново відкрити методом `open()` або перевстановити вказівник на початок:

```
f.seek(0)
```

2) Читання з файлу по рядках (спосіб 1):

```
for line in f:  
    ... # line - поточний рядок файлу, разом з '\n'
```

Тобто, змінна `line` по чергово прийматиме значення рядків файлу. Наприклад, якщо у файлі `data.txt` містяться числа (по одному на рядок), то їхню суму можна порахувати так:

```
s = 0  
with open('data.txt') as f:  
    for line in f:  
        s += float(line.strip())
```

або, використовуючи генератор¹:

```
with open('data.txt') as f:  
    s = sum(float(line.strip()) for line in f)
```

¹ Генератори розглядатимуться пізніше, але з їхнього синтаксису можна і зараз зрозуміти принцип їх роботи

3) Читання з файлу по рядках (спосіб 2):

```
with open('data.txt') as f:
    while True:
        line = f.readline()
```

4) Створення списку рядків файлу:

```
with open('result.txt') as f:
    lines = f.readlines()
```

```
# У змінній lines буде список ['Квадрати перших десяти натуральних чисел:\n',
'1\t4\t9\t16\t25\t36\t49\t64\t81\t100\t']
```

Режими відкриття файлів

Крім розглянутих вище трьох режимів відкриття файлів (для запису, для додавання в кінець і для зчитування), які задаються значеннями аргументів 'w', 'a' і 'r' відповідно, існує ще режим створення нового файлу для запису (якщо файл з вказаним іменем вже існує, то буде виведено повідомлення про помилку). Йому відповідає значення 'x'.

Під час роботи з нетекстовими файлами (зображеннями, аудіозаписами та ін.), потрібно вказувати, що вони повинні відкриватися у двійковому режимі. Це здійснюється додаванням символу 'b' перед потрібним режимом: 'bw', 'ba', 'br', 'bx'.

Імена файлів. Шляхи

Імена файлів 'data.txt' і 'result.txt', які використовувалися вище, є короткими іменами файлів. Приймаючи їх, функція open() шукає файли з такими іменами у поточній папці, тобто в тій же папці, у якій збережена програма. Для встановлення зв'язку з файлами, збереженими в інших місцях файлової системи, використовуються абсолютні або відносні імена файлів (шляхи). Елементи шляху відокремлюються символом '/' (для операційних систем Linux і macOS) або '\' (Windows). Щоб уникнути керівних символів у шляху, потрібно використовувати екранування або неформатовані рядки.

Абсолютне ім'я файлу містить шлях (ланцюжок папок) до вказаного файлу від кореневої папки (диску), відносно - від робочої директорії:

```
# абсолютні імена (шляхи):
'/home/user/code/numbers.txt'
'C:\\user\\code\\numbers.txt'      # використано екранування символів
```

```
r'C:\user\code\numbers.txt'          # використано неформатований рядок

# відносні імена (шляхи):
'../user/code/numbers.txt'          # . - поточна папка
'../user/code/numbers.txt'          # .. - батьківська папка для поточної
```

Модулі стандартної бібліотеки для роботи з файловою системою

os - модуль для роботи з операційною системою

os.path - підмодуль модуля os для роботи з шляхами

shutil - модуль для роботи з файлами та директоріями

Приклад застосування:

```
import os

paths = [item for item in os.listdir()
         if os.path.isdir(item)
         and not item.startswith('.')]


```
